



A20OS：高性能混合内核操作系统

2026 全国大学生计算机系统能力大赛

OS 内核实现赛道·初赛进展汇报

AAAAAAAAAAAAAAAAAAAAA

武汉大学

2026/6/30

自强
弘毅
拓新



目录

- ① 项目概览
- ② 核心架构
- ③ 技术特色
- ④ 设计文档详解
- ⑤ 测试验证
- ⑥ 未来计划

自强
弘毅
求是
拓新

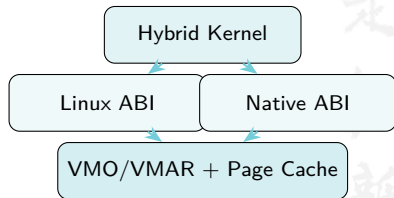
1 项目概览

自强弘毅
求是拓新



A200S 是什么？

- 混合内核：宏内核执行效率 + 微内核逻辑抽象
- 双重 ABI：高保真 Linux 兼容 + 面向未来的 Native ABI
- 跨平台：QEMU 四架构 + VisionFive 2 / LS2K1000
- 工程化：严格的锁契约、压力测试与多架构构建门禁



核心目标

探索现代操作系统架构边界，在兼容性、并发安全与内存效率上取得平衡。

初赛完成度速览

维度	状态	关键指标
内核代码规模	完成	约 12.5 万行 / 533 个源文件
支持架构	完成	RISC-V / ARM64 / x86_64 / LoongArch
Linux ABI 兼容	基本完成	musl / git / vim / gcc / fastfetch / mksh
SMP 并发	完成	NR_CPUS > 1 通过压力门禁
Native ABI	完成	90 个原生 syscall 入口
网络栈	完成	lwIP + virtio-net 事件驱动

核心原创代码位于 `kernel/`，用户态工具与第三方协议栈已明确区分。

求是 自强 弘毅 拓新

代码分布与工程规模

模块	代码行数
arch / HAL	30k+
mm / 内存管理	22k+
fs / 文件系统	18k+
proc / 调度	15k+
abi / 系统调用	14k+
drivers / 驱动	12k+
net / 网络	8k+
core / 基础设施	6k+

工程亮点

- 统一 Makefile 跨架构构建矩阵
- 文档化锁契约与并发门禁
- 内核/外部代码严格目录隔离

② 核心架构

自强弘毅
求是拓新



双重 ABI 架构

Linux ABI 兼容层

kernel/abi/linux/

- 高保真 POSIX/Linux 系统调用
- openat2、renameat2 复杂边界语义
- 高级 Futex、调度策略与亲和性

Native ABI 能力层

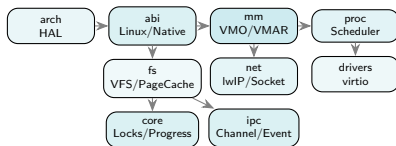
kernel/abi/native/

- 基于 Capability 的对象句柄
- VMO/VMAR、Channel、EventQ
- 90 个原生 syscall 入口

设计价值

既保留 Linux 用户态生态 (musl、git、vim)，又为下一代安全内核 API 提供演进底座。

内核模块组织



- kernel/core/: 锁层级契约、时间 keeping、panic 与 progress 机制
- kernel/include/core/lock.h: 全局锁顺序 `cg_node.lock` -> `proc_lock` -> `runq_lock`

求是
自強
拓弘
毅新



锁契约: SMP 并发的基础

```
1  /* Global lock order contract */
2  cg_node.lock -> proc_lock -> runq_lock
3  proc_lock -> files_struct.lock
4  proc_lock -> mm_struct.lock
5  proc_lock -> a20_handle_table.lock
6  g_lwip_lock -> g_net_lock
```

- 摒弃全局大锁，确立严格局部锁层级
- CONFIG_DEBUG_LOCKS 记录 owner/spinner 信息
- 新锁必须在 docs/driver-lock-order.md 文档化

核心规则

Never acquire proc_lock while holding runq_lock; never block while holding a spinlock; never call VFS/mm/scheduler while holding device or lwIP locks unless documented nonblocking.

③ 技术特色

特色一：Capability-based Handle 能力系统

- 所有底层资源统一为 对象句柄
- 每个 Handle 附带细粒度 rights 掩码
- 支持 时间到期 与 操作次数限制
- 集成 Bell-LaPadula 多级安全标签
- 防止 IPC 传递中的能力逃逸

关键文件：

kernel/abi/native/handle_table.c

```
1 | typedef struct a20_handle_entry {  
2 |     void          *object;  
3 |     uint16_t      type;  
4 |     a20_rights_t   rights;  
5 |     uint64_t      expiry_tick;  
6 |     uint32_t      remaining_ops;  
7 |     uint8_t       security_label;  
8 |     uint8_t       state;  
9 | } a20_handle_entry_t;
```

求是 自强
弘毅 拓新



特色二：VMO/VMAR 对象化内存模型

- **VMO** (Virtual Memory Object): 物理页容器, 可被 resize、共享、按页 fault
- **VMAR** (Virtual Memory Address Region): 地址空间布局, 负责映射与权限
- 将物理内存与虚拟地址布局 **解耦**, 为零拷贝 IPC 提供底座

关键文件: kernel/mm/a20_vmo.c、
kernel/mm/a20_vmar.c

```
1 | typedef struct a20_vm_object_args {  
2 |     uint32_t size;  
3 |     uint32_t version;  
4 |     uint64_t size_bytes;  
5 |     uint32_t flags;  
6 |     a20_handle_t out_handle;  
7 | } a20_vm_object_args_t;
```

```
1 | a20_vmo_create(size, &vmo);  
2 | a20_vmar_map(vmar, vmo, addr,  
3 |             len, prot, &mapped);  
4 | a20_vmar_protect(vmar, addr, len, prot);
```

求是
自強
弘毅
拓新



特色三：SMP 分级调度器

- **Per-CPU** 独立运行队列，消除全局调度锁
- **8 级优先级** + bitmap 0(1) 快速选任务
- 支持 **老化提升** (aging promotion) 与 SCHED_FIFO/RR
- 与 cgroup CPU quota 联动，实现 **限流调度**

关键文件：kernel/proc/sched.c、kernel/proc/cg_cpu.c

```
1 | proc_runq_t rq[NR_CPUS];  
2 | uint8_t bitmap[SCHED_LEVELS];  
3 | sched_promote_aged_locked(rq);
```

自
强
弘
毅
求
是
拓
新

特色四：大页降级、COW 与 MAP_SHARED 一致性

- `mm_demote_huge_page()`: 将 2 MiB 大页安全拆分为 4 KiB 页
- `mm_fork_clone_present_level()`: fork 时建立 写时复制 映射
- `mm_sync_shared_dirty_for_vnode()`: 维护 MAP_SHARED 脏页生命周期
- `mm->lock` 锁模型保证多核 mmap/munmap/fault 并发安全

关键文件: `kernel/mm/vm.c`、`kernel/include/mm/vm.h`

```
1 | typedef struct mm_struct {  
2 |     spinlock_t lock;  
3 |     vm_area_t *mmap;  
4 |     uint64_t *pgdir;  
5 |     refcount_t refcount;  
6 |     size_t total_vm, rss;  
7 | } mm_struct_t;
```

自強弘毅
求是拓新



特色五：事件驱动网络栈

- lwIP 以 NO_SYS=1 模式集成，避免多线程轮询
- kernel/core/progress.c 提供统一的 **bottom-half** 进展机制
- virtio-net RX/TX 由 **中断 + 事件** 驱动
- 调度器切任务前运行 bottom-half，降低上下文切换开销

关键文件：kernel/net/lwip_stack.c、kernel/core/progress.c

调用路径

网卡中断 → a20_lwip_poll() → progress_run() → socket bottom-half → 唤醒等待任务

求是
自強
弘毅



特色六：高兼容 VFS 与路径解析

- 实现 `openat2` 语义：`RESOLVE_NO_XDEV`、`BENEATH`、`NO_SYMLINKS`
- 正确处理 `mount root`、`..` 穿越、符号链接深度限制
- 支持 `ramfs/devfs/procfs/sysfs/cgroupfs` 等伪文件系统
- `Page Cache` 与 `block cache` 统一，保证多核一致性

关键文件：`kernel/fs/vfs/path_resolution.c`、`kernel/fs/page_cache.c`

求是
自强
弘毅
拓新

特色七：Native IPC —Channel 与 EventQ

- `a20_channel_create` 创建双向消息通道
- 单次消息最多携带 **64 KiB 数据 + 8 个 Handle**
- `a20_event_wait` 实现异步事件多路复用
- Handle 共享语义原子化，避免能力逃逸

关键文件：`kernel/ipc/a20_channel.c`、`kernel/ipc/a20_event.c`

```
1 | #define A20_CH_MAX_DATA      65536
2 | #define A20_CH_MAX_HANDLES  8
3 |
4 | typedef struct a20_msg_send_args {
5 |     a20_handle_t channel;
6 |     uint64_t      data;
7 |     uint32_t      data_len;
8 |     uint64_t      handles;
9 |     uint32_t      handle_count;
10 |    uint64_t      transfer_rights;
11 | } a20_msg_send_args_t;
```

自強弘毅
求是拓新

特色八：用户态生态与关键应用移植

- **vim 9.2.0433**：静态链接，termcap stub 替代 ncurses
- **git 2.54.0**：关闭 SSL/CURL/ICONV/PTHREADS，纯静态构建
- **gcc 14.2.0**：基于 musl-cross-make 的交叉/宿主工具链
- **fastfetch 2.62+**：无 CMake 手写 config header，禁用 dlopen

运行基础：Linux ABI + musl libc + lp64

构建入口

```
make -C user ARCH=riscv64
make -C user ARCH=riscv64
fastfetch
make -f user/extra.mk
ARCH=riscv64
make ARCH=riscv64
run-riscv64-extra
```

特色九：Native ABI 设计哲学

- 系统调用压缩：223 个 Linux 系统调用 → 90 个 Native syscall（约 2.5 倍压缩）
- 统一抽象：handle_set_meta 替代 chmod/chown/utimes/truncate；
handle_transfer 统一 splice/sendfile/copy_file_range/tee
- 版本化 **ABI**：每个复杂结构以 size + version 开头，支持运行时协商
- 无 fork/exec/signal/ioctl；以 task_spawn、Channel、EventQ 替代

关键文件：kernel/abi/native/syscall_table.def、
kernel/include/abi/native/types.h

求是
自強
弘毅
拓新

特色十：Native 启动协议与运行时

- 内核通过寄存器传递
a20_start_info_t
- 初始 Handle 显式声明：root_dir、
cwd_dir、stdio、self_task、
event_queue
- 三层运行时：Kernel → liba20rt →
liba20c → POSIX shim
- musl 移植：仅替换 arch 层与 syscall
ops，复用 ~45K 行代码

关键文件：user/liba20rt/crt0_*.S、docs/native-abi/07-startup.md

```
1 // liba20c: fd <-> handle mapping
2 #define A20_FD_INIT_SIZE 32
3 #define A20_FD_MAX_SIZE 1024
4
5 // musl target triple
6 aarch64-unknown-a20elf
```

自強弘毅
求是拓新



特色十一：安全模型—rights lattice

- **14 位权利掩码**：READ / WRITE / EXEC / STAT / DUP / TRANSFER / MAP / WAIT / CONNECT / ACCEPT / CONTROL / ADMIN / SIGNAL
- **权利单调递减**：handle_dup 与 IPC 转移只能缩小权利，不能扩大
- **Bell-LaPadula** 多级安全标签 {L, M, H}：no-read-up / no-write-down
- **Temporal capability**：到期时间与操作次数限制，自动回收或惰性失效

关键文件：docs/native-abi/06-security.md、kernel/abi/native/handle_table.c

特色十二：文件系统后端契约矩阵

后端	rename	link	symlink	关键限制
FAT32	×	×	×	元数据仅存 RAM
ext4	✓	×	快链 $\leq 60B$	无日志, nlink=1
ramfs	✓	✓	✓	max inodes 4096
pipe	—	—	—	PIPE_BUF 原子写

关键文件：docs/fs-consistency-model.md、docs/vfs-edge-semantics.md

求是
自强
弘毅
拓新

特色十三：运行时网络配置

- 无编译期网络默认值：旧硬编码 QEMU 地址已移除
- 内核命令行：a20.ip、a20.netmask、a20.gateway、a20.dns、a20.dhcp、a20.hostname
- 运行时暴露：/proc/net/config 只读展示当前生效配置
- 缺失配置时 socket 返回 -ENETUNREACH，而非静默使用开发默认值

关键文件：kernel/net/net_config.c、docs/network-config-design.md

自強弘毅
求是拓新



特色十四：混合驱动模型

- 零开销 **MMIO**：板级地址用宏内联，`readl/writel` 编译为单条 `load/store`
- 统一 **hwapi**：`request_irq`、`dma_alloc`、`clock_get_cycles` 跨架构抽象
- 类 **ops vtable**：`block_dev_ops_t`、`net_dev_ops_t`、`char_dev_ops_t`
- 编译期注册：`DRIVER_REGISTER` / `BOARD_REGISTER` 放入 `.driver_init` linker section

关键文件：`docs/driver-interface.md`、`kernel/include/drivers/hwapi.h`

求是
自強
弘毅
新



4 设计文档详解



自强弘毅
求是拓新





Native ABI: 从 223 到 90 的系统调用压缩

- 保留 POSIX 语义核心:
open/read/write/close、mmap/munmap、clone/wait/exit
- 用对象句柄统一替换 fd 语义:
handle_set_meta 替代
chmod/chown/utimes/truncate
- 用 handle_transfer 统一
splice/sendfile/copy_file_range/tee
- 取消 fork/exec/signal/ioctl, 以
task_spawn + Channel + EventQ 替代

类别	Native syscall 数
Task & Process	12
Memory (VMO/VMAR)	14
IPC (Channel/EventQ)	16
IO & Handle	22
Time & Sync	10
Misc	16
Total	90

关键文件: docs/native-abi/01-overview.md、kernel/abi/native/syscall_table.def



Handle 生命周期：从创建到回收

- 创建：对象构造 → 分配 entry → 初始化 rights / expiry / label
- 共享传递：IPC msg_send 时源表保留、目标表新增，对象引用计数递增，权利只能缩小
- 复制：handle_dup 可在同进程内复制，rights 子集
- 回收：关闭引用 → entry 置空 → 对象引用计数归零后释放

```
1 | typedef enum {
2 |     HANDLE_STATE_EMPTY    = 0,
3 |     HANDLE_STATE_ALLOC    = 1,
4 |     HANDLE_STATE_VALID    = 2,
5 |     HANDLE_STATE_EXPIRED  = 3
6 | } a20_handle_state_t;
```

不变式

任意时刻，一个 Handle 在同一进程 Handle Table 中只存在一份有效引用；IPC 共享传递是原子的，不存在“悬挂引用”。接收方 HT 空间不足时整体返回 NO_SPACE，消息留在队列中，杜绝 partial delivery。

关键文件：docs/native-abi/03-handles.md、kernel/abi/native/handle_table.c



VMO/VMAR: 解耦物理页与地址空间

- `a20_vmo_create(size, flags, &h)`: 创建物理页容器
- `a20_vmar_map(vmar, h, offset, len, prot, &addr)`: 将 VMO 映射到地址空间
- 保护公式: $\text{effective_prot} = \text{vmo_prot} \cap \text{vmar_prot} \cap \text{map_prot}$
- 共享映射: 多个 VMAR 映射同一 VMO, 实现零拷贝共享内存

关键文件: docs/native-abi/04-memory.md、kernel/mm/a20_vmo.c、
kernel/mm/a20_vmar.c

```
1 // VMO resize / commit / decommit
2 a20_vmo_set_size(h, new_size);
3 a20_vmo_op_range(h, COMMIT, off, len);
4 a20_vmo_op_range(h, DECOMMIT, off, len);
5
6 // Shared zero-copy IPC
7 a20_vmo_create(PAGE_SIZE, 0, &vmo);
8 a20_vmar_map(sender_vmar, vmo, ...);
9 a20_channel_write(channel, ..., &vmo, 1);
10 a20_vmar_map(receiver_vmar, vmo, ...);
```

Channel/EventQ: 异步 IPC 协议

- 两阶段写入: reserve 槽位 → 拷贝数据
→ commit; 失败可 abort
- **Handle** 共享传递: 消息 commit 后源
进程仍保留 Handle, 对象引用计数递增
- **EventQ**: 每个进程一个, Channel 就绪
时通过 a20_event_signal 通知
- 等待: a20_event_wait 多路复用, 支持
deadline 和可中断

```
1 // Two-phase channel write
2 a20_channel_call_args_t args = {
3     .channel = ch,
4     .data = buf, .data_len = n,
5     .handles = &h, .handle_count = 1,
6 };
7 a20_channel_call(&args, &reply);
```

保证

消息要么完整交付 (数据 + 所有 Handle), 要么完全不交付; 不存在半包或 Handle 重复。

关键文件: docs/native-abi/05-ipc.md、kernel/ipc/a20_channel.c



安全模型: rights lattice 与 MLS

- 14 位权利掩码构成格, ADMIN 为最大元, 空权利为最小元
- 单调性: $\text{rights}(\text{child}) \subseteq \text{rights}(\text{parent})$
- **Bell-LaPadula**: 低密级主体不可读高密级对象 (no-read-up), 高密级主体不可写低密级对象 (no-write-down)
- **Temporal**: expiry_tick 与 remaining_ops 提供自动失效

关键文件: docs/native-abi/06-security.md

标签	语义
L (Low)	普通用户数据
M (Medium)	系统服务中间层
H (High)	内核关键资源

求是
自強
弘毅
拓新



Native 启动协议与 musl 移植

- 内核通过 a0 寄存器传递
a20_start_info_t *, 包含初始 Handle
和配置
- 初始 Handle: root_dir、cwd_dir、
stdin/stdout/stderr、self_task、
event_queue
- 运行时分层: liba20rt (syscall 封装)
→ liba20c (fd 映射、pthread shim) →
musl
- musl 移植: 新增 arch 层 a20elf, 替换
syscall 入口, 复用 stdio/string/malloc
等 ~45K 行

关键文件: docs/native-abi/07-startup.md、user/liba20rt/crt0_*.S

目标三元组

aarch64-unknown-a20elf
riscv64-unknown-a20elf
x86_64-unknown-a20elf
loongarch64-unknown-a20elf

VFS 边界语义与一致性契约

- `openat2` 标志: `RESOLVE_NO_XDEV`、`RESOLVE_BENEATH`、`RESOLVE_NO_SYMLINKS`
- `renameat2`: `RENAME_EXCHANGE`、`RENAME_NOREPLACE`、跨 mount 拒绝
- 路径解析: 处理 mount root、.. 穿越、符号链接深度 `MAX_SYMLINKS=40`
- Page Cache: `writeback` 与 `invalidate` 保证 `MAP_SHARED` 多核一致

关键文件: `docs/vfs-edge-semantics.md`、`docs/fs-consistency-model.md`

关键不变式

`lookup` 必须持有父目录引用;
`rename` 原子更新目录项与 inode nlink;
跨后端操作返回 `-EXDEV`。

网络配置与驱动接口设计

- 无编译期默认网络参数，全部通过内核命令行注入
- 命令行参数：`a20.ip=10.0.2.15`、`a20.netmask=255.255.255.0`、`a20.gateway=10.0.2.2`、`a20.dns=10.0.2.3`、`a20.dhcp=off`、`a20.hostname=a20os`
- `/proc/net/config` 只读暴露当前配置
- 驱动接口：`hwapi.h` 统一 `request_irq`、`dma_alloc`、`clock_get_cycles`

关键文件：`docs/network-config-design.md`、`docs/driver-interface.md`

设计原则

静默默认值是隐蔽 bug 的温床；缺失配置时 socket 显式返回 `-ENETUNREACH`。

求是
自強
弘毅
新





5 测试验证

自强弘毅
求是拓新





测试门禁与初赛成绩

测试目标	命令示例	状态
多架构内核构建	make check-kernel-build	通过
调度并发压力	make smoke-sched-stress	通过
Futex 并发压力	make smoke-futex-stress	通过
VFS 并发压力	make smoke-vfs-stress	通过
MM 压力与 COW	make smoke-mm-stress	通过
Linux ABI smoke	make smoke-riscv64	通过
并发基础门禁	make check-concurrency-foundation	通过

测试日志位于 test-results/，记录构建与运行时回归结果。

自
强
弘
毅

求
是
拓
新



P0 完成情况

已完成的 P0 方向

- SMP 并发与锁契约
- Linux ABI 正确性
- 高级 Futex 与内存序
- MM / Page Cache / MAP_SHARED
- Huge-page demotion / COW
- OOM reclaim 安全策略
- vim / git / gcc / fastfetch 移植与静态构建

Remaining P1/P2

- 块/网络设备完全事件驱动唤醒
- g_lwip_lock 竞争优化
- VFS 边界语义持续收紧
- 更多 LTP 风格压力测试

求是
自強
弘毅
拓新





下一步工作

- ① I/O 完全事件驱动：替换块/网络设备中的兼容轮询路径
- ② 网络栈深化：扩展 socket 并发测试，降低 lwIP 全局锁竞争
- ③ VFS 语义收紧：完整 renameat2、statx、xattr 等边界行为
- ④ 测试强化：将静态文档门禁转化为可执行的行为测试
- ⑤ 架构探索：在 riir 分支继续 Rust 重写实验（研究性质）

核心追求

在保证 Linux 生态兼容的前提下，构建更安全、更高性能、更可维护的下一代内核底座。

求是 自強 弘毅 拓



致谢

感谢全国大学生计算机系统能力大赛平台
感谢指导教师与开源社区的支持

Q & A

项目地址: <https://gitlab.eduxiji.net/T202610486999803/oskernel2025-a20>
GitHub 镜像: <https://github.com/fQwQf/A200S>

自强弘毅
求是拓新